E4 Guided troubleshooting: who does what

Support desk backend · version 2, 4 November 2026 · S1 and S2 are built on `feature/E4-guide-runs` (schema at Flyway V10)

What changed since version 1. S1 and S2 are built and tested, and the contract below is now the real one. Six things differ from version 1:

1. Start takes `{ticketNumber, guideId}`, not `guideVersionId`. The server picks the guide's live version, and it only accepts a guide that fits the ticket.
2. Previous is `DELETE /api/guide-runs/{id}/answers/{position}`, not `POST /back`. Sending it twice changes nothing.
3. `INVALID_GUIDE_ROUTE` is 422, not 400, like the codebase's other rule errors.
4. A run stores only its answers. There are no `status`, `current_step_key` or `outcome_key` columns: the step, the outcome and the status are replayed from the answers against the pinned version.
5. The entities are `GuideRun` (field `pinnedVersion`) and `GuideRunAnswer`, over the `troubleshooting_sessions` and `troubleshooting_answers` tables.
6. Matching lives on `guides` plus two small tables, not a `guide_match_rules` table. A ticket whose error code is only its category's default does not match or hint on it.

1. Split at a glance

Owner	Stories	Scope	Branch
Youssef	E4-S1 Recommend guides E4-S2 Run a guide step by step	**Built.** Guide schema and seed, matching, runs and answers, runner API (start, view, answer, back), log hints.	`feature/E4-guide-runs`
Ahmed	E4-S3 Outcome and customer script	Finish endpoint, outcome handler (resolve / escalate / follow-up), run summary, `GUIDE_COMPLETED`, auto-logged customer script.	`feature/E4-guide-outcomes`
Mohab	E4-S4 Version pinning E4-S5 Flag a step	Pinning tests and the saved-run view, `GuideFeedback` table, flag endpoint, Guide Admin feedback queue.	`feature/E4-guide-feedback`

S4 is mostly proving what S2 already stores (a run keeps its `guide_version_id` and can never be re-pointed), so it pairs with S5 to even out the load. The runner **screen** (tech task 6) is frontend work. It is not assigned here because the frontend has not started; it consumes the API in section 3.

2. Order of work

Step	What	Who	Unblocks
1	**Merge** `feature/E4-guide-runs`: V10, the guide package, the seed, the permission rules, the error codes, Swagger and the Postman folder "8. Guided troubleshooting (E4)".	Youssef	Ahmed and Mohab branch from `main` after it merges
2	S3 finish endpoint, built on `GuideRun.status()` and the outcome in the run view.	Ahmed	S5 (a finished run)
2	S4 tests and the S5 feedback table and admin queue. Flagging needs a completed run, so test it with a run finished through S3 or completed in the test.	Mohab	None

3	Rebase on `main`, run the full suite against Postgres and add the Postman requests.	Everyone	None

3. Shared contract (as built)

Tables (V10)

Table	Columns that the others rely on
`guides`	`id, title` (unique), `category_id` (the sub-category it troubleshoots)
`guide_platforms`, `guide_error_codes`	`guide_id, platform` and `guide_id, error_code`. The platform must match. Then an error-code match ranks above a category match.
`guide_versions`	`id, guide_id, version_number, state` (`DRAFT · IN_REVIEW · LIVE · RETIRED`), `tree jsonb, published_at`. At most one LIVE version per guide (partial unique index). Published trees are never edited; a change is a new version (E7).
`troubleshooting_sessions`	`id, ticket_id, guide_version_id` (not updatable), `started_by_id, started_at, completed_at`. One run per ticket with `completed_at IS NULL` (partial unique index). **Ahmed adds the summary and outcome columns in V11.**
`troubleshooting_answers`	`session_id, position` (1-based, unique per run), `step_key, answer_key, answered_at`. Previous deletes from a position onwards.
`tickets` (new column)	`error_code_from_category`: true when the ticket's error code is only its category's default. `Ticket.customerErrorCode()` then returns null, so guides neither match nor hint on it.

Tree JSON stored in `guide_versions.tree`

```
{
  "start": "app-version",
  "steps": {
    "app-version": {
      "question": "Is the app on version 5.2 or later?",
      "answers": [ { "key": "yes", "label": "Yes", "next": "other-device" },
                   { "key": "no",  "label": "No",  "outcome": "update-app" } ],
      "hints": [ { "errorCode": "ERR_AUTH_503", "answerKey": "no", "text": "The login service did not
answer." } ]
    }
  },
  "outcomes": {
    "update-app": { "type": "RESOLVED", "result": "The app was updated and login works.",
                    "customerScript": "Please update the app from the store, then log in again." }
  }
}
```

Every answer has exactly one of `next` or `outcome`. Outcome `type` is `RESOLVED`, `ESCALATE` or `FOLLOW_UP`. The `GuideTree` record rejects a tree with a missing step, a missing outcome, a dead end or a loop when it is built, so code holding one never re-checks routes. The server is the only judge of a route: it replays the run against its pinned version.

Endpoints

Endpoint	Does	Owner
`GET /api/guides/recommended?ticket={number}`	Live guides that fit the ticket, error-code matches first, each with `matchedBy` and a `matchLabel` such as "Matches error ERR_AUTH_503". Empty means the ticket's next step is Escalate.	Built

`POST /api/guide-runs {ticketNumber, guideId}`	Starts the live version, moves the ticket to TROUBLESHOOTING and audits `GUIDE_STARTED` with the version. 409 if the ticket has an unfinished run; 422 `GUIDE_DOES_NOT_FIT` for a guide that does not fit.	Built
`GET /api/guide-runs/{id}`	The run view: `status`, `path`, `currentStep` (question, options, hint, stepNumber, maxSteps) and `outcome` (type, result, customerScript) once reached.	Built
`POST /api/guide-runs/{id}/answers {stepKey, answerKey}`	Answers the current step; moves to the next step or to `OUTCOME_REACHED`. A stale step or an unknown answer is 422 `INVALID_GUIDE_ROUTE`.	Built
`DELETE /api/guide-runs/{id}/answers/{position}`	Previous: back to the step answered at that position, clearing it and every later answer. Works from an outcome too.	Built
`POST /api/guide-runs/{id}/complete {notes?}`	Applies the outcome to the ticket, saves the summary, sets `completedAt` and audits `GUIDE_COMPLETED`.	Ahmed
`POST /api/guide-runs/{id}/flags {stepKey, comment}`	Flags a step on a completed run.	Mohab
`GET /api/admin/guide-feedback`	The Guide Admin feedback queue (paged, newest first).	Mohab

Answer and Previous lock the run row, so two tabs on one run cannot interleave their answers.

Shared files: who edits what

File	Change	Owner
`Permissions` + `PermissionMatrixTest` AREAS	**Done:** `/api/guides/**` and `/api/guide-runs/**` for AGENT. `/api/admin/**` already covers the feedback queue.	Youssef
`AuditAction` + constraint migration	Each person adds their value and widens `ck_audit_logs_action` in their own migration: `GUIDE_STARTED` (V10, done), `GUIDE_COMPLETED` (V11), `GUIDE_STEP_FLAGGED` (V12).	Each
`ErrorCode`	**Done:** `GUIDE_NOT_FOUND` and `GUIDE_RUN_NOT_FOUND` (404), `GUIDE_RUN_ALREADY_ACTIVE` and `GUIDE_RUN_NOT_ACTIVE` (409), `INVALID_GUIDE_ROUTE` and `GUIDE_DOES_NOT_FIT` (422). `GUIDE_OUTCOME_NOT_REACHED` is Ahmed's.	As listed
`NextAction` / `TicketMapper` / `TicketQueryService`	**Done:** NEW or REOPENED with no fitting live guide resolves to ESCALATE instead of START_GUIDE.	Youssef
`TicketWorkspaceService`	Fill the `TODO (E4 guides)`: log the outcome's script as "Told customer: ...".	Ahmed

Migration numbers are reserved: V10 Youssef (done), V11 Ahmed, V12 Mohab. Merge in that order. If S4 needs no table, Mohab still uses V12 for the feedback table. Flyway rejects a lower number than one already applied, so never merge V12 before V11.

4. Per person: what "done" means

Youssef E4-S1 Recommend guides · E4-S2 Run a guide step by step Built

- Three local seed guides: "Login failed: server error" (retired v1, live v2, draft v3, for ERR_AUTH_503), "Login failed: general checks" and "Payment failed" (PAY_TIMEOUT).
- Only LIVE versions are recommended. The platform must match, then the error code or the category; error-code matches come first and are named.
- No match gives an empty list, and the ticket's next step reads Escalate, on both the ticket and the queue.
- Start moves the ticket to TROUBLESHOOTING and audits `GUIDE_STARTED` with the guide and version, in one transaction. A resolved ticket gets a 422 and nothing is recorded.
- One unfinished run per ticket. If the ticket was moved on by hand (resolved, then reopened), starting again closes the old run first.
- Hints show only for the customer's own error code, given on the call or found in their logs.
- Tests: 4 tree unit tests and 13 integration tests against Postgres. The full suite is 180 tests, and the Postman collection is 62 requests; both pass.

Ahmed E4-S3 Outcome and customer script

- The run view already carries the outcome's type, result text and "Say to the customer" script once reached.
- `complete` works only when `run.status() == OUTCOME_REACHED`. Otherwise it returns 409 `GUIDE_OUTCOME_NOT_REACHED`. Load the run with `GuideRunRepository.findLockedById`, like answer and Previous.
- **RESOLVED**: the ticket moves to RESOLVED.
- **FOLLOW_UP**: the ticket moves to WAITING_ON_CUSTOMER. `Ticket.moveTo` already pauses the SLA.
- **ESCALATE**: the escalation dialog and Azure work item are E5-S1. Until E5 lands, the ticket moves to ESCALATED through the system path and the run keeps its path for E5 to send. Leave a `TODO (E5 Azure DevOps)`.
- V11 adds the summary columns (outcome key, duration, notes) to `troubleshooting_sessions`. Set `completedAt`, and audit `GUIDE_COMPLETED` in the same transaction.
- Until S3 lands, a run that reaches an outcome stays open, and the ticket cannot start another guide while it is in Troubleshooting.
- The customer script is auto-logged to `ticket_contacts` as "Told customer: ..." (fills the existing TODO).
- Tests: one per outcome type, plus completing twice and completing before an outcome.

Mohab E4-S4 Version pinning · E4-S5 Flag a step

- **S4:** an integration test starts a run on the live version, then publishes a new one mid-run (inserts a LIVE row and retires the old one). It checks that the run finishes on the old questions and route. `GuideRun.pinnedVersion` is not updatable, so this should already hold.
- **S4:** a saved run always shows its own version number and steps, including after the guide is retired.
- **S5:** V12 `guide_feedback`: `session_id, guide_version_id, step_key, agent_id, comment, created_at, status` (`OPEN` for now; E7 handles review).
- **S5:** flagging is allowed on COMPLETED runs only, for a step on the run's path (`run.walk().taken()`). A blank comment is rejected with a 400 field error (`@NotBlank`).
- **S5:** the admin queue shows guide, version, step, agent and time. Audit `GUIDE_STEP_FLAGGED`.
- Tests: pinning, blank comment, a flag on an unfinished run, and the queue visible to GUIDE_ADMIN with a 403 for AGENT.

5. Conventions every PR follows

- Use the layered packages, as in `common`: new code goes under `guide/` with `controller, dto, entity, enums, repository, service`. Do not create feature sub-folders such as `guide/runner`.
- Each controller implements an `…Api` interface that holds the mappings and all Swagger annotations. Use `@PreAuthorize` on controllers as well as the `Permissions` rule.
- Use Lombok and Jakarta validation annotations, with DTOs as records. Errors go through `ErrorCode` and `ApiException`, so the body stays `{code, message, fieldErrors, path, timestamp}`.
- Write the audit in the same transaction as the change. `AuditService` is `MANDATORY`, so call it from a `@Transactional` service.
- Add your requests to the Postman folder "8. Guided troubleshooting (E4)", with tests for each new endpoint. Swagger must show every operation, and `SupportDeskApplicationTests` lists every tag.
- Integration tests run against the docker Postgres (`docker compose up -d`, port 5434). The full suite passes before review.

6. Open questions for the team

- **Escalate before E5:** is moving to ESCALATED with no work item acceptable for the demo, or should Escalate stop at OUTCOME_REACHED and wait for E5?
- **Progress:** is "step n of the longest path" what the design shows, or a plain answered count?
- **Abandoned runs:** as built, an unfinished run stays resumable with no timeout. It is closed only when its ticket has left Troubleshooting and a guide is started again. Is that enough, or should an agent be able to drop a run and pick another guide mid-call?
- **Runner screen:** who owns it, and when the frontend starts.